

BioChroma : Simulation bactérienne en milieu fermé

Étudiant : Axel Pigeon

Formation : Master Interactions des Mathématiques et de l'Informatique pour l'IA (IMA)

Date : 24 septembre 2026

Dépôt : Dépôt GitHub

1. Présentation générale

Motivation. Intéressé par la modélisation stochastique, j'ai cherché un projet mettant en commun l'efficacité du langage C++ et la simulation aléatoire. J'ai donc opté pour une simulation d'une marche aléatoire multi-agents sur une surface finie en 2D.

Généralités. *BioChroma* est un simulateur stochastique modélisant l'évolution d'une population de *bactéries* (agents) dans une *boîte de Pétri* (milieu). L'objectif est d'implémenter différents types de bactéries (héritage) permettant la mise en place d'interactions (prédateur/proies) entre les différents agents.

Principaux Objets. Chaque agent est modélisé par un objet autonome régi par des transitions markoviennes entre différents états (actif, en division, spore ou mort) en fonction de son niveau d'*énergie* et des *ressources* disponibles. La boîte de pétri est modélisée par une grille 2D où chaque *cellule* contient une certaine quantité de *nutriments* (ressources). À chaque étape de la simulation, une étape de *diffusion* des ressources dans le milieu est opérée avant la mise à jour de chaque agent.

Interface. *BioChroma* est pensé pour être un outil simple sans interface graphique poussée. L'initialisation de la grille se fera par le remplissage d'un tableau dans un fichier `.txt`. Pendant la simulation un affichage ASCII sera produit en ligne de commande. Enfin, un module d'exportation au format CSV sera disponible pour effectuer des analyses plus poussées sur la trajectoire de la simulation. Un exécutable en ligne de commande `kallisto` sera développé pour instancier la simulation.

2. Objectifs techniques

Ici sont décrits quelques détails techniques relatifs à l'implémentation du projet en C++ ainsi qu'aux attendus techniques de celui-ci.

- **Gestion dynamique de la mémoire :**

- La grille de nutriments (`Grid`) encapsule un buffer contigu alloué dy-

namiquement sur le tas (`float* nutrients` via `new[]` et `delete[]`). Les opérations relatives à la gestion mémoire seront respectées (constructeur, destructeur, copie profonde et surcharge d'opérateur).

- **Hierarchie de classes et polymorphisme :**

Une classe de base abstraite `Bacterium` déclare des méthodes virtuelles pures (`update`, `clone`) et un destructeur virtuel. Deux sous-classes concrètes spécialisent le comportement des agents :

- `StandardBacterium` : marche aléatoire isotrope, métabolisme de base et mise en état de dormance (`SPORE`) en cas de famine locale.
- `ChemotacticBacterium` : déplacement orienté par le gradient local de nutriments (chimiotactisme) avec une mobilité accrue.

- **Orchestration et sémantique d'accès :**

La classe `Simulation` gère le cycle de vie d'une collection hétérogène via des pointeurs polymorphes (`std::vector<Bacterium*>`). Les surcharges d'opérateurs sont exploitées pour les types d'algèbre 2D (`Vector2`).

- **Chaîne de compilation et tests :**

Un `Makefile` complet gère la compilation séparée en C++. Un module de test sera aussi développé et branché sur le `Makefile` via un `make test`.

3. Architecture globale

```
classDiagram
    class BacteriumState {
        <<enumeration>>
        ALIVE
        DEAD
        DIVISION
        SPORE
    }

    class Vector2 {
        +float x
        +float y
        +Vector2()
        +Vector2(float x, float y)
        +Vector2(const Vector2& other)
        +operator=(const Vector2& other) Vector2&
        +operator+(const Vector2& other) const Vector2
        +operator*(float scalar) const Vector2
    }

    class Bacterium {
```

```

    <<abstract>>
    #Vector2 position
    #BacteriumState state
    #float energy
    #float velocity
    #float mutation_prob
    #uint32_t id
    +Bacterium()
    +Bacterium(const Bacterium& other)
    +operator=(const Bacterium& other) Bacterium&
    +virtual ~Bacterium()
    +virtual update(float dt, float local_nutrient)* void
    +virtual clone()* Bacterium*
}

class StandardBacterium {
    +StandardBacterium()
    +StandardBacterium(const StandardBacterium& other)
    +operator=(const StandardBacterium& other) StandardBacterium&
    +~StandardBacterium() override
    +update(float dt, float local_nutrient) override void
    +clone() override Bacterium*
}

class ChemotacticBacterium {
    -float sensing_radius
    +ChemotacticBacterium()
    +ChemotacticBacterium(const ChemotacticBacterium& other)
    +operator=(const ChemotacticBacterium& other) ChemotacticBacterium&
    +~ChemotacticBacterium() override
    +update(float dt, float local_nutrient) override void
    +clone() override Bacterium*
}

class Grid {
    -int N
    -float cell_size
    -float* nutrients
    +Grid(int n, float cell_sz)
    +Grid(const Grid& other)
    +operator=(const Grid& other) Grid&
    operator()(int x, int y) const float
    +~Grid()
    +diffuse(float dt, float diffusion_coeff) void
    +get_nutrient_at(Vector2 pos) const float
    +consume_nutrient_at(Vector2 pos, float amount) void

```

```

}

class PetriDish {
    -Grid grid
    -float diffusion_coeff
    +PetriDish()
    +PetriDish(const string& filepath)
    +PetriDish(const PetriDish& other)
    +operator=(const PetriDish& other) PetriDish&
    +~PetriDish()
    +update(float dt) void
    +get_grid() const Grid&
}

class Logger {
    -string output_dir
    -uint32_t log_interval
    +Logger(const string& dir, uint32_t interval)
    +Logger(const Logger& other)
    +operator=(const Logger& other) Logger&
    +~Logger()
    +log(uint32_t step, const Simulation& sim) void
    +export_to_csv() void
}

class Simulation {
    -vector~Bacterium*~ bacteria
    -PetriDish environment
    -float dt
    -uint32_t current_step
    +Simulation(const PetriDish& dish, float dt)
    +Simulation(const Simulation& other)
    +operator=(const Simulation& other) Simulation&
    +~Simulation()
    +step() void
    +launch(uint32_t total_steps, Logger& logger) void
    friend class Logger
}

Bacterium <|-- StandardBacterium : inherits
Bacterium <|-- ChemotacticBacterium : inherits
PetriDish *-- Grid : contains
Simulation *-- PetriDish : contains
Simulation o-- Bacterium : manages (pointers)
Simulation ..> Logger : uses / friend
Bacterium ..> BacteriumState : uses

```

Bacterium ..> Vector2 : uses